

Agent Systems 研究型进阶专题 A

第 20 讲：课程结束，论文开工

From Course Completion to a Reproducible Paper Project

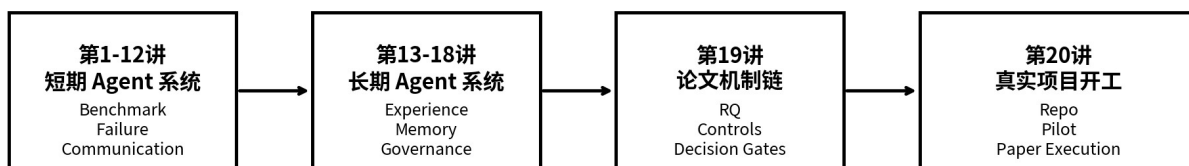
专题 A 收官：从今天开始，不再以“继续学习”替代真实实验

本讲的目标不是让你“准备得更充分”，而是把研究问题冻结到足够明确，让 Codex 和你可以直接开始做 Smoke Test。第 20 讲结束以后，不再继续第 21 讲；下一阶段只有真实科研执行。

1. 为什么现在必须停止继续学

前 19 讲已经把 Benchmark、Trace、Failure、控制实验、错误相关、通信、长期经验、共享记忆、治理、污染与恢复串成一套完整的方法论。继续增加知识点的边际收益已经低于“开始真实实验并从真实 Failure 中学习”的收益。科研基本功不是把地图背完，而是能完成一个最低闭环：现象 → 假设 → 对照 → 数据 → 分析 → 否决/继续。

- 以后遇到新概念：只在真实实验需要时补，不再预先把整个领域学完。
- 以后修改研究设计：必须由真实 Trace、失败模式或文献证据触发，而不是因为“又想到一个变量”。
- 第一篇论文的核心贡献只允许围绕机制链，不追求复杂大系统或 SOTA。



学习阶段结束的判据：不再因为“还没学完”而推迟真实实验

最小闭环：Preflight → Smoke → Pilot Gate → Confirmatory → Analysis → Paper

图 1：专题 A 从学习阶段切换到真实论文项目。

2. 第一篇论文到底要证明什么

研究对象固定为：相同基础 Agent 在长期经历不同 experience streams 后，是否形成可复现的功能性异质性；这种异质性是否改变错误结构，并在固定聚合规则下带来集体收益。

层级	核心问题	主要证据
Manipulation	Memory 是否真的被检索/使用？	retrieval log；Memory-Off
Causality	差异是否跟着 Experience Bank，而不是 Agent ID？	Memory-Swap
Heterogeneity	是否出现 family-specific specialization？	Agent × Task-family matrix
Error Structure	是否减少共同失败？	pairwise ϕ ；double-fault
Collective Outcome	固定 majority 是否获得额外收益？	majority error；collective gain
Boundary	全量共享会不会重新耦合错误？	Full-Share boundary

论文 Claim Stack：下层不成立，就不能跳到上层

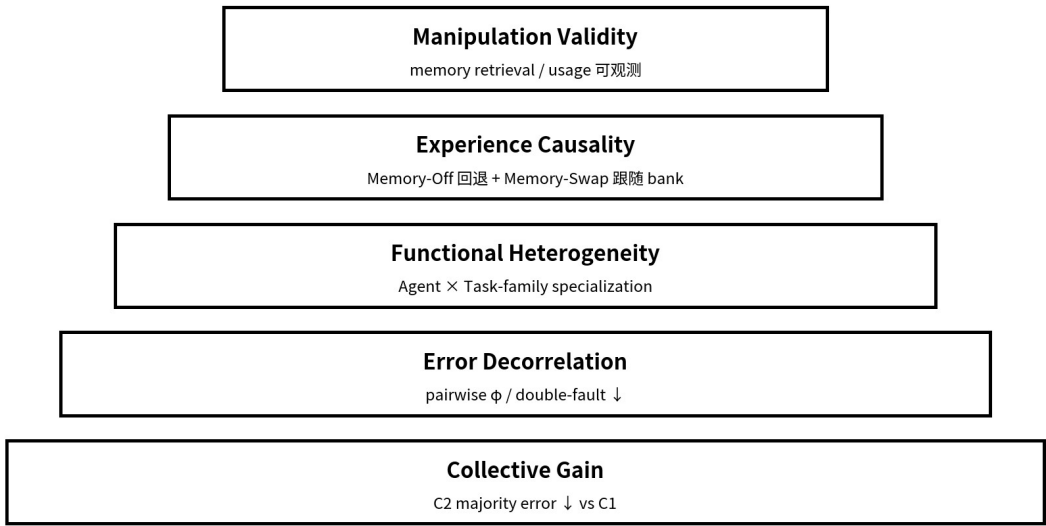


图 2：论文 Claim Stack。下层证据不足时，禁止跳到更强的上层结论。

3. 暂定论文标题、RQ 与贡献边界

3.1 暂定标题（工作标题，不是最终投稿标题）

Experience Divergence Induces Functional Heterogeneity and Error Decorrelation in Persistent LLM Agent Teams

3.2 Research Questions

- 1. RQ1：不同长期经历是否会在控制模型、Prompt、Role、Tool、Memory Budget 后形成稳定的功能性异质性？
- 2. RQ2：功能性异质性是否会降低 Agent 间错误相关与 double-fault？
- 3. RQ3：错误去相关是否能在固定 aggregation 下转化为 collective gain？

4. RQ4：关闭、交换或全量共享 memory 时，上述现象是否按机制预期回退或转移？

3.3 不能声称什么

- 不能声称“多 Agent 普遍优于单 Agent”。
- 不能仅凭 ϕ 与 collective gain 同步变化，就声称完成了严格 causal mediation。
- 不能把 Persona / Model / Role 差异当成 experience-induced heterogeneity。
- 不能把 Teaching Mock 或少量 cherry-picked case 当作真实论文证据。

4. 五个核心条件 + 一个边界条件

Condition	Memory / Experience	因果角色
C0 No Memory	无长期经验	基础能力基线
C1 Shared Experience	三个 Agent 使用相同经验库	控制 “有记忆但不分化”
C2 Divergent Experience	P/E/R 三条不同长期经历	核心处理组
C3 Memory-Off	与 C2 同 history，但关闭 retrieval	经验依赖性回退
C4 Memory-Swap	交换 P/E/R experience banks	专长是否跟随 bank
C5 Full-Share Boundary	C2 后全量共享 experience	错误重新耦合边界

第一篇论文不要把第 16-18 讲所有治理、污染、自愈模块都塞进核心实验。它们更适合作为后续工作、边界讨论，或下一篇论文。第一篇必须窄。

5. 真实科研仓库：每个文件都要有“证据身份”

本讲资料包内已经给出 project/experience_divergence_paper/。核心原则是：代码、配置、原始输出、人工标注、统计结果和论文图表分离，禁止手工改结果 CSV 后直接画图。

```
experience_divergence_paper/  
├ configs/      # 冻结版本、模型、任务、条件、seed、预算  
├ schemas/      # run / memory / judgment / failure 表结构  
├ src/          # preflight、规划、Gate、审计与分析代码  
├ outputs/      # 机器生成；不得手工改数字  
├ paper/        # 论文 outline、Figure/Table 规划  
├ checklists/   # Smoke / Pilot / Confirmatory / Submission  
└ codex/        # Codex 总提示词与任务板
```

6. 12 周计划：把“3 个月内第一篇”变成 Deadline

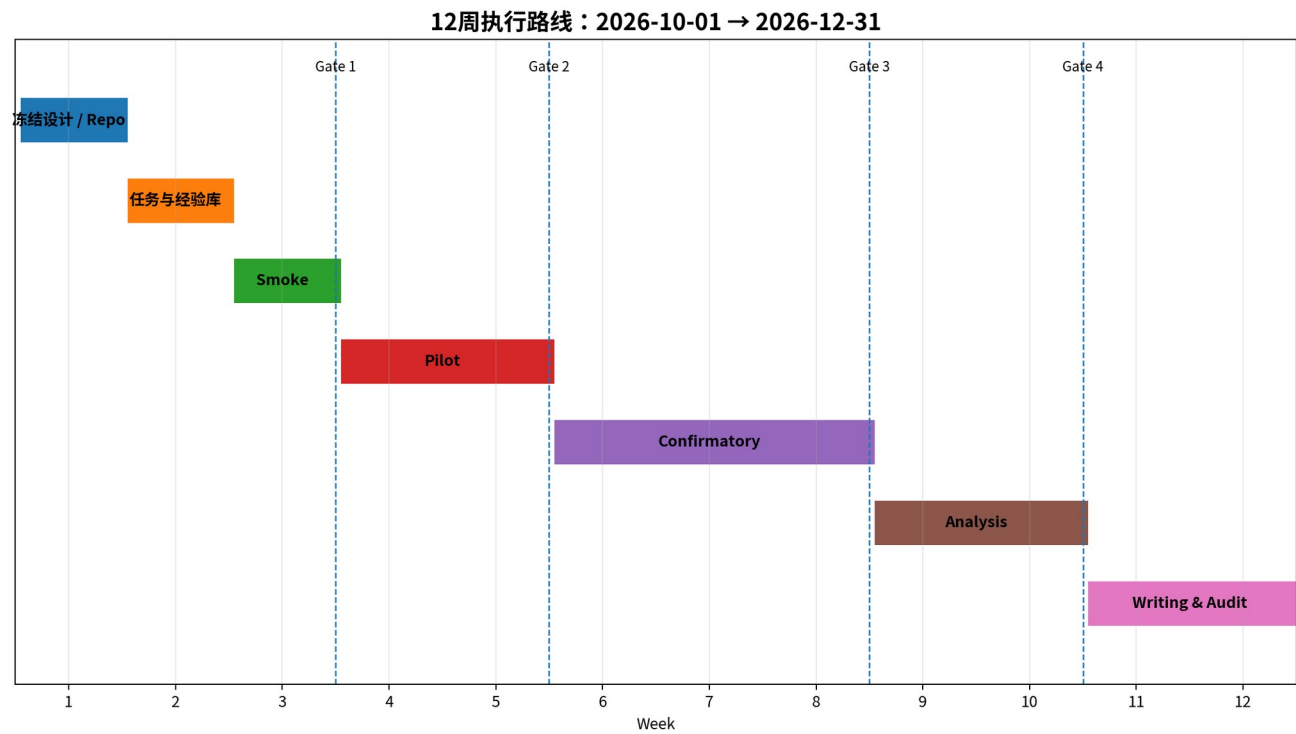


图 3：从 2026-10-01 到 2026-12-31 的 12 周执行路线。

阶段	周	必须产物	进入下一阶段的条件
Freeze	W1	Repo + prereg + task taxonomy	所有变量/版本可复现
Data/Experience	W2	经验库 + held-out split	无泄漏、数量平衡
Smoke	W3	6-10 tasks	管线/日志/评分全部正确
Pilot	W4-5	10 tasks × 2 trials	4 个核心 Gate 方向成立
Confirmatory	W6-8	约 30 held-out tasks	不再按结果调假设
Analysis	W9-10	CI / Failure / figs	数字可由脚本重建
Writing/Audit	W11-12	完整稿 + reproduce audit	第三方按 README 可复跑

7. Gate 比“计划”更重要：什么时候立即停止

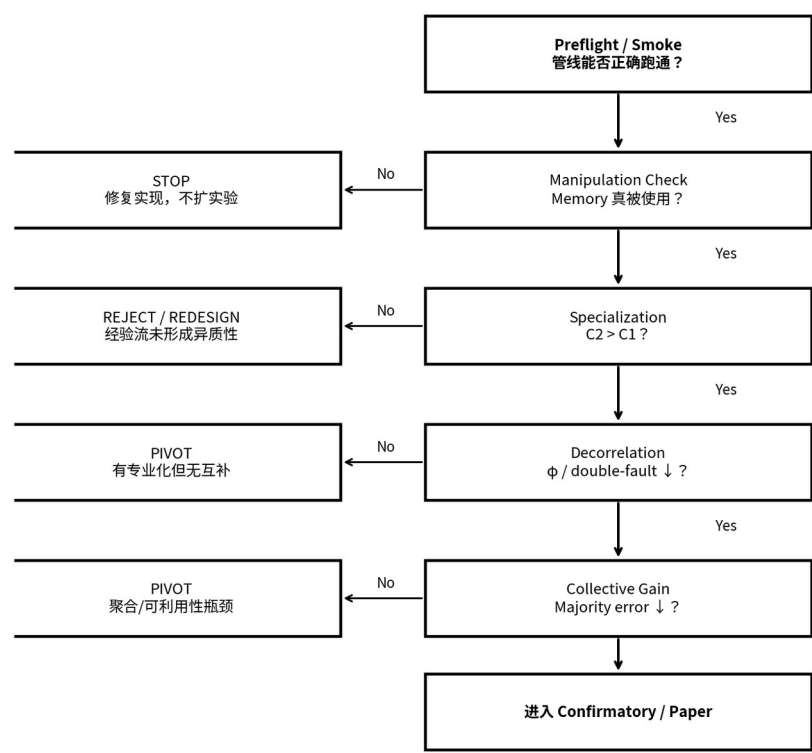


图 4：实验决策树。Gate 失败时先否决或转向，不继续堆预算。

第 20 讲给你的最重要科研习惯，是把“停止条件”提前写下来。结果不好并不可怕；没有停止规则、一路调参数直到出现想要结果，才会让项目失去科学性。

8. 成本与资源控制：先测单条成本，再决定规模

- Smoke Test 先测真实平均 agent_cost、user_cost、latency 和失败率。
- 任何大批量任务前先计算 projected_cost = 实测单条成本 × 计划 trajectories。
- 如果 Confirmatory 预算超过预设 ceiling，先减少不必要条件/模型或调整 task 数；不要静默超预算。
- Pilot 只决定“值不值得扩大”，不是用来反复调到显著。

9. Codex 在这个项目里是什么角色

Codex 是实验工程执行者，不是研究问题的最终裁判。你和我负责 RQ、控制变量、证据标准和否决规则；Codex 负责仓库搭建、脚本实现、测试、批量运行、日志检查和可复现报告。

可以交给 Codex	不能直接交给 Codex 决定
搭环境、写 runner、写单元测试	修改核心 RQ
批量实验、断点续跑、日志/manifest	看到结果后偷偷换 primary metric
生成统计表和图的代码	删掉“不好看”的失败任务
检查 schema / duplicate / leakage	把相关性写成因果结论
资料包已提供 CODEX_MASTER_PROMPT.md。真实开工时，先把整个项目目录交给 Codex，让它只完成 Phase 0/1，不要一次要求它自动跑完整个 900-trajectory Confirmatory。	

10. 本专题的最低闭环能力：你现在应该能独立判断什么

- 一个 Agent Benchmark 是否真的测到了你关心的行为，而不是只看 leaderboard。
- 一次失败是偶然、基础设施错误、稳定 failure pattern，还是系统机制问题。
- 一个“新方法”到底改变了哪个变量，是否存在 hidden confound。
- 多个 Agent 的价值来自成员强度、错误互补还是错误传播。
- 长期 memory / sharing / governance 如何改变系统错误结构。
- 实验结果什么时候值得扩展，什么时候应该否决原假设。

11. 最后一条规则：第 20 讲以后不要继续第 21 讲

下一步不是再学习一门“Agent Systems 高级课”，而是打开 `project/experience_divergence_paper/`，执行 `00_preflight.py`，冻结 manifest，然后开始 Smoke Test。以后所有学习都围绕真实项目遇到的问题按需补齐。

专题 A 到此结束。真正的第 21 讲应该由真实实验结果来决定，而不是由课程大纲来决定。